



Exact algorithm for matrix-based project planning problems



Zsolt T. Kosztján

Department of Quantitative Methods, University of Pannonia, Hungary

ARTICLE INFO

Article history:

Available online 4 February 2015

Keywords:

Project planning methods
Decision-making tools
Exact algorithms
Supporting traditional and agile project managements

ABSTRACT

This paper proposes a new matrix-based project planning method that takes into consideration task importance or probability of completions thus determines and ranks the importance or probability of possible project scenarios and project structures. The proposed algorithm is fast, aims to select the most important project scenarios or the least cost/time demanding project structures. The algorithm is generic, can host several types of goals dictated by the characteristics of project management and as such can be the fundamental element of a project expert- and decision-making system.

© 2015 Elsevier Ltd. All rights reserved.

1. Introduction

The most frequently used, traditional project planning methods, e.g. network-planning methods (Eisner, 1962; Kelley, 1961; Roy, 1962; Wiest, 1981), Gantt charts (Wilson, 2003), Line of Balance methods (Badukale & Sabihuddin, 2014) primarily support the operative tasks of project planning. Having an accepted logic plan, where tasks and the dependencies between them are determined, we can schedule tasks and allocate costs and resources by using different kinds of network planning, cost and resource allocation methods (Brucker, Drexler, Mohring, Neumann, & Pesch, 1999). Having an accepted logic plan we can (only) schedule tasks and allocate costs and resources by using different kinds of network planning, cost and resource allocation methods (Brucker et al., 1999). Since the tasks and dependencies between them are already determined these algorithms not really support decision-making problems like:

- Which tasks or subprojects should be selected for a new project?
- Not having a strict technological order of tasks, which is the best possible sequence of tasks regarding time, resource and cost demands of the project?

Evaristo and van Fenema (1999) demonstrated that the implemented resource-allocation and project planning methods were usually developed for construction projects. According to Wysocki's (Wysocki, 2009) project managers' survey, projects supported by so called traditional project management (TPM) give nowadays only 20% of the whole running projects. Methods for investment and construction projects usually cannot be used

directly for software development or R&D projects managed by agile project management approaches. However, in the practice, multiple project environments (MPE) exist where more than one project is managed simultaneously (Hashim, Chileshe, & Baroudi, 2013). In MPE several (running) projects can be supported by traditional projects management techniques, at the same time others should be handled with agile approach.

Supporting the logic planning Kosztján and Kiss (2013) proposed a matrix-based method instead of traditional project planning techniques, aiming to meet management claims. The proposed matrix-based methods support not only traditional but agile project management.

A number of researchers Baker and Freeland (1975), Carbonell, Rodriguez Escudero, and Munuera Aleman (2004), Coldrick, Longhurst, Ivey, and Hannis (2005), Gidel, Gautier, and Duchamp (2005), Hart, Jan Hultink, Tzokas, and Commandeur (2003), Linton, Walsh, and Morabito (2002), Krishnan and Ulrich (2001), and Rochford (1991) have studied project scoring and ranking methods. However, most of the methods are rather too simple to deal with complex decision making problems or too complex (see e.g. Chin, Xu, Yang, & Ping-Kit Lam, 2008; Liao, 2005; Monghasemi, Nikoo, Fasaee, & Adamowski, 2015; Oh, Yang, & Lee, 2012) to guarantee the optimal solution.

This paper proposes a novel fast, exact generic algorithm to select and order the top n possible project plans with respect to time/cost/resource constraints. The proposed generic algorithm support not only the traditional but also the agile project management. Projects can be restructured and reorganized respect with the changes of the management claims.

This paper has five sections. After the introduction, literature review on the agile vs traditional project management, network-based vs matrix-based project planning, project selection and ranking methods and project expert system are presented. The third

E-mail address: kzst@vision.vein.hu

section presents the theoretical background and details of the proposed method. The fourth section presents the simulation results. It is followed by the conclusion section which summarizes the research findings respectively.

2. Related works

In this section, the difference of the traditional and the agile project management approaches will be presented first. The agile project management (APM) approach is mainly used for software development projects (SDPs) (Dingsøyr, Nerur, Balijepally, & Moe, 2012), however, this approach can also be applied for new product development (NPD) projects (Fekri, Aliahmadi, & Fathian, 2009; Rahimian & Ramsin, 2008).

Network-based project planning and resource allocation methods are applied in traditional Project Management System (PMS), however, APM may require new project planning methods (see Section 2.1). Therefore in the following subsection matrix-based project planning methods will be presented. The last two subsections will focus on the project selection methods and project expert and decision support systems.

2.1. Agile vs traditional project management

The two approaches of project management on strategic level: the traditional and the agile project management are illustrated on Fig. 1. In case of traditional project management there is a scope which has to be achieved within a given time and with a certain budget, but time and cost can vary according to requirements. The objective function could be the minimal total cost, balanced resource demands, or minimal project duration (see Brucker et al., 1999). In agile project planning deadlines, resource and cost availability are fixed (see Dalcher, 2009). The goal could be the realization of as much tasks as possible.

In case of construction projects the question is how much it will cost for all these features? So, the scope is given, which has to be completed, but time and cost can be changed if it is necessary. In the agile approach the question is how many of these features can be included within this budget and time interval? The goal is to realize the scope at the highest possible degree. The agile project planning approach can be widely used, for example, in case of software development projects (SDPs) (Chan & Thong, 2009), and in case of new product development project (NPD). There are numerous literatures (see i.e. Dingsøyr et al., 2012; Williams & Cockburn, 2003), which form the principles of the agile approach (simplicity, communication, etc.). Estimating SDPs will always be a fraught

activity; users frequently change their minds, things often do not work out as planned. Agile estimation is much easier and more accurate because the iterations of the estimation are based on a good understanding of the work.

In SDPs the technological order is less important than in classic (e.g. construction) projects. So, the sequence of tasks may be reversed and the SDPs can be restructured.

The Software Project Scheduling Problem (SPSP) is a specific Project Scheduling Problem (PSP) (Chen, 2011; Laalaoui & Bouguila, 2014) that supports SDPs, however, the reviewed methods cannot cover the APM approach. SPSP consists the appropriate worker-task assignment that minimizes cost and duration for the whole project, so the task precedence and resource constraints are satisfied (Alba & Chicano, 2007; Chang, Jiang, Di, Zhu, & Ge, 2008; Nan & Harter, 2009). The SPSP (also) organizes multiple activities to meet the defined objectives, to these activities renewable or non-renewable resources can be assigned.

While Software Project Scheduling Problem is one of the most common problems in managing software engineering projects, this model cannot support the agile approaches (Özdanar & Ulusoy, 1995). In the agile approach the feedback is very important, but SPSP assumes the logic plan is modeled as a direct acyclic graph. Although, cycles and feedbacks can be modeled by GERT method (Pritsker, 1966), but it is difficult to find cycles in a complex network. The other problem is the fixed dependencies of tasks in the SPSP are given, however, in case of the APM approach the project plan may be restructured and/or reorganized with respect to the changes of management claims.

In APM the time, cost and resources are also restricted conditions, but the goal of specifying logic planning is to find the optimal project plan with respect to the order of tasks, which are within these constraints.

The traditional project planning methods based on network planning techniques assume that a project plan (see Kelley, 1961; Roy, 1962), or the alternatives of possible projects (Eisner, 1962; Pritsker, 1966) are specified and should not be modified or respecified. However, the proposed matrix-based method allows the project manager to select the most desired project plan and restructure or reorganize the project if the customer claims change.

2.2. Evaluation of the matrix-based project planning methods

Matrix-based methods, along with network planning techniques, are mainly used for planning and scheduling production development projects according to Danilovic and Browning (2007). These matrix methods are based on the DSM (Design

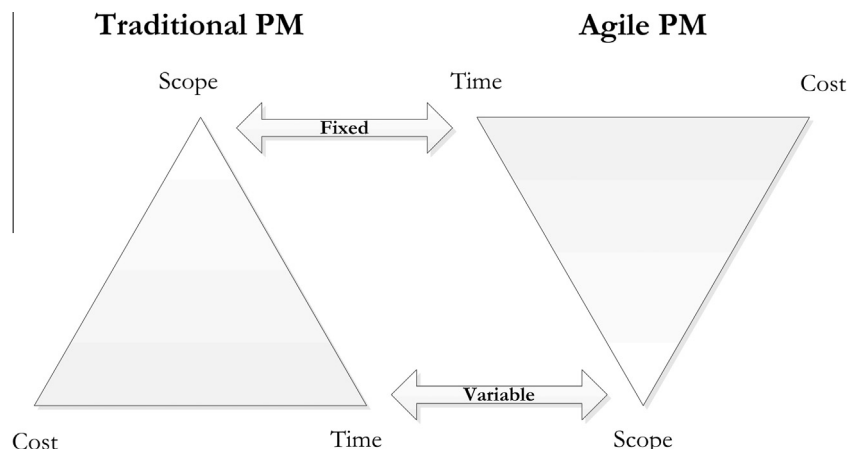


Fig. 1. Difference between traditional and agile project management (PM) approaches.

Table 1

Basic relations between two tasks (dsmweb.org).

Sequential relation	Parallel relation	Coupled (iterated) relation

Structure Matrix/Dependency Structure Matrix) methods published by [Steward \(1981\)](#). Project tasks are represented both in the rows and columns of a matrix. DSM method was originally employed for system and architecture modeling where the acronym of DSM means Design Structure Matrix, but in case of project planning the acronym means Dependency Structure Matrix. [Eppinger, Whitney, Smith, and Gebala \(1994\)](#) improved the original DSM method for modeling relations between two tasks therefore this DSM method is suitable for project planning. DSM method specifies three basic relations between two tasks. These relations can be sequential, parallel and coupled (iterated). Relations are shown in an adjacency matrix (see [Table 1](#)) where “X” specifies the (deterministic) relation between two tasks.

Although loops or iterated relations between two tasks were discussed in an early publication [Pritsker \(1966\)](#) such relationships hardly integrated into standard project management tools, whereas matrix-based methods are able to handle loops and cycles. Using sequencing algorithms feedback, loops can be detected ([Xiao, Chen, & Tao, 2007](#)). Formerly introduced methods show how to use matrix-based methods for project planning. However, DSM matrices can also be used for scheduling ([Chen, Ling, & Chen, 2003](#); [Minogue et al., 2011](#)) and resource allocation ([Browning, 2014](#); [Browning & Eppinger, 2002](#); [Yan, Wang, & Jiang, 2002](#)).

According to [Danilovic and Browning \(2007\)](#), one way of extending DSM method is to specify multiple domains (e.g. tasks, resources etc.). Since the traditional DSM focuses on a single domain, its analysis yields insights deemed optimal from the point of view of that domain only (e.g. dependency between tasks). However, the Domain Mapping Matrix (DMM) provides means to represent interactions and relations between domains (e.g. task-resources: Who will do these tasks?).

The main deficiency of DSM and DMM is that they mark only the evidence of strict relations but these do not provide any special information about the relation itself. Further researches ([Tang, Zhu,](#)

[Tang, Xu, & He, 2010](#); [Yassine, Falkenburg, & Chelst, 1999](#)) showed, that uncertain relation can be specified between two tasks. This method, the so-called Numerical DSM (NDSM) can handle the strength of the relation between two tasks. When using NDSM matrices the level of dependency of relations between two tasks can be represented and numbers stand for “X” s in NDSM ([Chen et al., 2003](#); [Tang et al., 2010](#)) i.e.:

- Dependency strength: (this) can be a measured between 0 and 1, where 1 represents an extremely strong dependency. The matrix can now be partitioned by minimizing the sum of the dependency strengths below the diagonal.
- Probability of repetition: (this) number reflects the probability of one activity causing rework in another. Feedback relationships represent the probability of having to loop back (i.e. iteration) to earlier (upstream) activities after a downstream activity was performed, while feed-forward relationships can represent the probability of a second-order rework following an iteration.

SDP and innovation project (IP) tasks can often be completed both in sequential and parallel order. This situation can be modeled by another matrix-based project planning method called Stochastic Network Planning Method (SNPM) introduced by [Kosztyán and Kiss \(2010b\)](#). This acronym indicates that one model can represent several possible network plans (see [Table 2](#)).

However, the NDSM method also contains stochastic relations between two tasks, but the SNPM method shows that there are two different possible project plans in case of an uncertain relation. These possible project plans (described as DSMs or project networks) called as (possible) *project structures*.

Similarly, to the DSM, 0 or empty cell means that there is no dependency between tasks. 1 or “X” stands for the strict dependency between two tasks, and a measure between 0 to 1 means *uncertain relations*. Uncertain relation can mean relative

Table 2

SNPM – sequential and parallel completion (“X” certain, “?” uncertain relation, dotted arrow denote uncertain relation).

SNPM matrix	possible DSM matrices	Possible project networks

Table 3

PEM – possible project scenarios and project plans (“X” certain, “?” uncertain completion/relation, dotted arrow denote uncertain relation, and dotted box denoted uncertain task completion).

PEM	SNPMs	DSMs	Project nets

importance of relation between two tasks, but uncertain dependency can be described as a probability of the relation between two tasks. In this case if the values of off-diagonal cells are probabilities of the relation instead of the importance, it is signed with $p_{A-B} \in [0, 1]$ in case of tasks A and B, then $1 - p_{A-B} \in [0, 1]$ shows the probability that there is no relation between the two tasks. If $1 - p_{A-B} = p_{A-B} = 0.5$, then the relation between these two tasks is indifferent, so the tasks can be realized sequentially and in parallel with the same probability. Determining the number of possible project plans is a combinatorial problem, because if there are k uncertain relations then there are 2^k possible project plans.

The relations between tasks can be treated as objective probabilities, if there is some prior information about possible technological order from previous realized similar projects (in this case they are objective probabilities), or further possible technological relations can be formed based on different experts' opinion (in this case they are subjective probabilities).

The other interpretation of uncertain relations is the relative importance level of the relations between two tasks. In this case the importance level of the relation between task A and task B shows how important it is that task A is followed by task B.

In this situation dependency strength is more appropriate instead of the probability of the dependency. Uncertainty value can be described as a score value, which can mean either a probability value or the strength of the dependency between two tasks.

SNPM method is extended by Kosztyán and Kiss (2010a), where probabilities or priorities can also be assigned to the completion of the tasks. If one cannot estimate the probability or priority of the completions/relations one can say the completions/relations are uncertain (denoted with “?” in Table 3). This representing method is the Project Expert Matrix (PEM).

Both PEM and SNPM matrices have two versions: *specified* and *semi-specified* matrices. Specified SNPM and specified PEM matrices contain specified score values within an $[0, 1]$ interval. The semi-specified SNPM (see Table 2) and semi-specified PEM (see Table 3) matrices contain strict = certain and/or flexible = uncertain dependencies between tasks or mandatory = certain and/or supplementary = uncertain task completions. However, the score values of uncertain task dependency/completion are non specified.

During the evaluation of PEM, the possible projects can be determined in two steps. First, one should decide which tasks should be executed. Table 3 shows that completion of Task B is uncertain. Therefore, two SNPM matrices or project scenarios can be defined, where Task B is executed and where Task B is omitted. The result is two *project scenarios*.

Project scenarios can be described as SNPM matrices (see Table 3). Secondly, if we decide that Task B will be completed, we should decide how to complete these project scenarios: parallel or sequential. So, we can say there are two possible project plans or *project structures*. If Task B is not completed, the question of ‘how to complete Task A and Task B’ will be irrelevant, because only one project structure exists where we complete only Task A (see Table 3).

In this case the main challenge is to determine possible project plans. If there are t uncertain task completions, the number of possible project scenarios is 2^t . The other main challenge is to determine feasible project scenarios and project structures regarding time, resource and cost constraints and ordering these feasible project plans by their probabilities/importance or according to other score values. Previously, solving this problem with fast exact algorithm was absolutely hopeless, thus genetic algorithms (Kiss, 2013; Kosztyán, 2012) were applied to handle this selecting and ranking problem. In this study new exact algorithms are introduced for selecting and ordering the first n feasible project plans.

In order to handle time, cost and resource demands and constraints, Project Expert Matrix (PEM) is proposed to extend to a new project domain matrix (PDM), similarly to extending DSM to DMM. DMMs are already used for modeling project management problems (see e.g. Browning, 2014; Danilovic & Browning, 2007), but the novel PDM can model also the uncertain task completions and uncertain task dependencies.

2.3. Project screening and selection methods

A project selection and project screening is a type of multi-criteria decision analysis (MCDA) problem. A number of researchers Baker and Freeland (1975), Carbonell et al. (2004), Coldrick et al. (2005), Gidel et al. (2005), Hart et al. (2003), Linton et al. (2002), Krishnan and Ulrich (2001) and Rochford (1991) have studied project scoring and ranking methods. However, most of the methods, are rather too simple to deal with complex decision making problems. Other methods (Chin et al., 2008; Monghasemi et al., 2015; Oh et al., 2012) tried to harmonize the different kinds of project demands and claims like: resources, cost demands, importance and relevance of the projects. Widely range of MCDA methods, like Analytic Hierarchy Process (AHP) (see Calantone, Di Benedetto, & Schmidt, 1999; Hsu, Tzeng, & Shyu, 2003; Mohanty, Agarwal, Choudhury, & Tiwari, 2005), Evidence Reasoning (ER) (see e.g. (Wang & Elhag, 2008; Yang, 2001) and the combination of AHP and ER (Chin et al., 2008)) are used for project selection problems. Such multiple criteria have to be considered and assessed at project screening stages, which involves a group of cross-functional experts. Due to a large number of quantitative and qualitative criterias and lack of sufficient and concrete data, group members often have to make decisions in uncertain situations.

These methods focus on the project scoring process. Usually the result is the ranked list of scored projects. However, when managing SDPs numerous scored project can be or can become an infeasible project. In case of APM smaller elements of project should be considered. While in TPM we focus on projects, the APM approach focuses on tasks and/or subprojects, because at the end of a sprint meeting also new tasks can be defined. In this case we should specify the list of task that can be completed within strict time and resource constraints. Infeasible project structures and project scenarios should be ignored, and at the project should be restructured and/or reorganized.

The proposed method can handle the score value of project completion and project dependencies, therefore project plans can be (re) structured with respect to the (changes of the) customer claims. Score values of task completion are used instead of project

scoring, however, AHP and ER or ER–AHP methods can also be used to specify task scores.

In order to satisfy also the TPM and the APM claims, the proposed method specify different kinds of score value for time, cost, resource demands and the importance of task completion. While importance of task completion can integrate different kinds of qualitative and quantitative criterias similarly to the applied AHP and ER and ER–AHP approaches, the resource, cost and time demands should be maintained separately.

2.4. Project expert system

The methodological aspect was in the 1950–60s' main stream of project management publications according (Crawford, Pollack, & England, 2006) was very strong, however, Kastor and Sirakoulis (2009) showed that very few results of these methods are implemented in PMS applications. For example, heuristic methods are used for resource allocation, and resource levelling; therefore different types of PMS lead to different results for the same resource-constrained scheduling problem. According to Lynn Crawford in the 1970s–1980s other aspects (like team-work, handling multi projects and project portfolios, connection between strategy and projects etc.) became more and more important, while unfortunately PMS applications very slightly supported these approaches. For instance, with using traditional, most frequently used PMS software (like: Microsoft Project, Primavera Project Planner), we can handle connections and resource sharing within a multi project or a programme, but we cannot determine and rank different project scenarios, we cannot assemble a project portfolios considering the company's strategy.

Knowledge-based techniques are important in project management. The first project expert systems were developed in the late 1980s (Arditi & Patel, 1989; Burger & Fischer, 1985). Until now numerous researchers have developed project expert system (ES) and decision support systems (DSS) for supporting different kinds of projects (Arditi & Patel, 1989; Burger & Fischer, 1985; Cho, 2006; Hadad, Keren, & Laslo, 2013; Mahdi & Alreshaid, 2005). While most of the published and developed ES and DSS concentrate on very important and risky issues like: project selecting, screening and risk a management problems, most of these systems with a few exceptions (see an exception: Boehm, Madachy, & Steece, 2000) are not really incorporated in the market place.

In this article only the structure of an expert system will be specified, however, the proposed method can be the fundamental basis of a project expert system. Different kinds of objective functions can be specified with respect to the management claims for selecting the adequate project plan. The proposed method supports the agile project management approach beside the traditional project management.

3. Selecting and ordering project scenarios and project structures

This section proposes a new matrix-based project planning method. The first subsection presents a novel project domain matrix (PDM). The following subsection introduces how to calculate score values of project scenarios from task importance or probability of completions. This subsection also shows how to determine the minimal and maximal time, cost and resource demands in order to exclude infeasible project scenarios and project structures.

3.1. Project domain matrix

A project domain matrix (PDM) is an n by m matrix, where n is the number of tasks; $m = n + t + c + r$; where t is the number of

possible duration, c is the number of possible (direct) cost and r is the number of possible resource demands of tasks.

PDM has four domains. The first domain is the logic domain (LD). That is described as an n by n PEM matrix (see PEM at Section 2.2). Since PEM has specified and semi-specified versions, the PDM is *specified* if and only if the LD is specified otherwise the PDM is *semi-specified*.

The second domain is the time domain (TD). That is an n by t matrix. Similarly to the early network-based techniques Kelley (1961) and Roy (1962), if every task duration estimated as a specified time demand, we say TD is *deterministic*, ($t = 1$). However, duration can be described by more than one parameters (see e.g. Eisner, 1962; Lootsma, 1997; Pritsker, 1966) in this case the t is the number of parameters of time demands (we say TD is *stochastic*). In this way we denote the minimal/maximal (possible) task durations as $t_{\min}/t_{\max}$.

Cost domain (CD) specifies an n by c matrix, where c represents the number of parameters of (direct) cost demands. The direct cost of a task usually characterized as only single value (we say CD is *deterministic*), however the cost demand of a task can be interpreted as a time–cost (De, Dunne, Ghosh, & Wells, 1995; Feng, Liu, & Burns, 2000), or a time–quality–cost trade-off (see e.g. Mungle, Benyoucef, Son, & Tiwari, 2013) function. In this case CD should contain the minimal and maximal cost demand of a task ($c = 2$). Similarly to the durations, costs can also be characterized with fuzzy techniques (we say CD is *stochastic*). In this case we denote the minimal/maximal cost demands of the task as $c_{\min}/c_{\max}$.

The last domain is (renewable) resource domain (RD). It is assumed, that every task has (at most) k renewable resource demand. Denote $r_1, r_2, \dots, r_k$ as the resource demands. If every resource demand can be characterized as a single value, we can say RD is *deterministic*, $r = k$. However, similarly to the costs, resource demands can also be characterized as a resource–time trade-off function or a resource–quality–time trade-off function and similarly to the time durations and cost demands, resource demands can also be characterized with fuzzy techniques (in this case we say RD is *stochastic*), we denote the minimal/maximal resource demands as $r_{i\min}/r_{i\max}$, where $i = 1, 2, \dots, k$.

We say PDM is *deterministic* if TD, CD and RD are deterministic, otherwise PDM is *stochastic*. With respect to the various kinds of domains four types of PDM: (1) specified deterministic, (2) specified stochastic, (3) semi-specified deterministic and (4) semi-specified stochastic can be determined.

Primarily specified deterministic PDM will be applied (Fig. 2) to demonstrate the project scoring and the proposed exact algorithm. However, this algorithm will be extended to maintain other kinds of PDMs.

It is worth to remark, that this method can also support the agile project management approach, where regarding budget, time and resource constraints as many task will be completed as possible. In traditional project management, there is no way to exclude tasks from a project, therefore, any value of the diagonal of the LD should be 1. However, also in case of traditional project management approach subprojects can be excluded or postponed to a later multi project.

3.2. Project scoring

Let us consider a specified deterministic n by m PDM matrix, where n is the number of tasks; $m = n + t + c + r$; t is the number of possible time, c is the amount of possible (direct) cost and r is the number of possible resource demands of the tasks. $t = c = 1, r = k$.

If a PDM is given, four different types of score value can be specified:

PDM	A	B	C	D	E	t	c	r_1	r_2	r_3
A	0.8	1	0.8	0.2	0.1	4	2.4	2	1	1
B		1		0.4	0.9	2	1.8	3		3
C			0.1		0.1	4	9.6	8	2	5
D				0	0.3	10	42	12	3	2
E					0.7	3	0.9	1	4	2
Logic Domain (LD)						TD	CD	Resource Domain (RD)		

Fig. 2. Specified deterministic project domain matrix.

- (1) Project score value describes either a possible project scenario or a project structure. This value can mean.
 - (a) Either probability value.
 - (b) or importance value.
- (2) Score value of task duration.
- (3) Score value of resource demands.
- (4) Score value of cost demands.

3.2.1. Calculation of score value of project scenarios and project structures

If uncertain task completion can be characterized as a probability value then probability value of a project scenario can be calculated as the product of the probability value of the task completions (see Eqs. (1) and (2)). If uncertainty values can be characterized in another way – as other score values – then total score value of a project scenario can be calculated and should be used to characterize project scenarios.

In the first case we assume that score values of a task completion and a task dependency mean a probability value. Later the proposed method will be extended to apply different kinds of score values.

The decision about task realization we always have two options: include or exclude. As a result the proposed method is a tree traversal algorithm. At a given node first we find whether the decision really free or locked, given the task is mandatory.

Let e_{ij} , where $i = 1, 2, \dots, n, j = 1, 2, \dots, m$ be a (cell) element of the **PDM** matrix. e_{ii} represents the probability value of completion of the task i . Although $1 - e_{ii}$ is the probability value of exclusion of the task i from the project.

At the initial step let us define two additional n by n matrices such as **P**, **Q**. In this step **P** represents the LD of the **PDM**, $Q = 1 - P$. p_{ij} is an element of **P**, where at the initial phase $p_{ij} = e_{ij}, i, j = 1, 2, \dots, n$. q_{ij} is an element of **Q**, where at the initial phase $q_{ij} = 1 - e_{ij}, i, j = 1, 2, \dots, n$.

If there are t uncertain tasks, then 2^t project scenarios can be specified. However, without determining any more possible project scenario, the probability of most probable

$$\left(P_{\max} = \prod_{i=1}^n \max(p_{ii}, q_{ii}) \right)$$

and less probable

$$\left(P_{\min} = \prod_{i=1}^n \min(p_{ii}, q_{ii}) \right)$$

project scenarios can be calculated in a very simple way. If we decide, that uncertain task i should be completed, e_{ii} will be modified to 1. In a very similar way, if we decide to exclude task i from the project, e_{ii} will be modified to 0. After every decision the most desired ($P_{\max} = \text{MAXSCORE}(\mathbf{LD}, \mathbf{P}, \mathbf{Q})$) and less desired

($P_{\min} = \text{MINSORE}(\mathbf{LD}, \mathbf{P}, \mathbf{Q})$) project scenarios can be calculated with respect to the former decisions about including/excluding tasks (see Eqs. (1) and (2)).

$$P_{\max} = \prod_{i=1, p_{ii} \neq 0}^n P_{i_{\max}}, \text{ where } P_{i_{\max}} = \begin{cases} p_{ii}, & \text{if } e_{ii} = 1 \\ q_{ii}, & \text{if } e_{ii} = 0 \\ \max(p_{ii}, q_{ii}), & \text{otherwise} \end{cases} \quad (1)$$

$$P_{\min} = \prod_{i=1, p_{ii} \neq 0}^n P_{i_{\min}}, \text{ where } P_{i_{\min}} = \begin{cases} p_{ii}, & \text{if } e_{ii} = 1 \\ q_{ii}, & \text{if } e_{ii} = 0 \\ \min(p_{ii}, q_{ii}), & \text{otherwise} \end{cases} \quad (2)$$

After deciding which tasks have to be completed, dependencies between tasks should be specified. In this case off-diagonal values mean probability of dependencies between tasks. The probability value of a possible project structure is calculated as a production of probability values of dependencies between included tasks (see Eqs. (3) and (4)).

$$p_{\max} = \prod_{i=1, e_{ii}=1}^n \prod_{j=1, p_{ij} \neq 0, i \neq j}^n p_{ij_{\max}}, \text{ where } p_{ij_{\max}} = \begin{cases} p_{ij}, & \text{if } e_{ij} = 1 \\ q_{ij}, & \text{if } e_{ij} = 0 \\ \max(p_{ij}, q_{ij}), & \text{otherwise} \end{cases} \quad (3)$$

$$p_{\min} = \prod_{i=1, e_{ii}=1}^n \prod_{j=1, p_{ij} \neq 0, i \neq j}^n p_{ij_{\min}}, \text{ where } p_{ij_{\min}} = \begin{cases} p_{ij}, & \text{if } e_{ij} = 1 \\ q_{ij}, & \text{if } e_{ij} = 0 \\ \min(p_{ij}, q_{ij}), & \text{otherwise} \end{cases} \quad (4)$$

In this section the specified PDM is assumed, however the proposed method can also be applied to unspecified PDM. In this case the score value of uncertain task completion and uncertain task dependency are specified as $p_{ij} = q_{ij} = 0.5$.

Besides probability values other score values like importance value can also be applied for characterizing the score value of task completion. For scoring task completion different kinds of scoring techniques, like AHP (see Calantone et al., 1999; Hsu et al., 2003; Mohanty et al., 2005), ER (see Wang & Elhag, 2008; Yang, 2001) or ER-AHP (Chin et al., 2008) can also be used.

If an aggregate function (like summation, production) is specified in order to calculate project scenario from the score values of task completion and other aggregate function is specified to calculate project structure from the score values of task dependencies, the most desired project scenario ($P_{\max}$)/project structure ($p_{\max}$) and the less desired the project scenario ($P_{\min}$)/project structure ($p_{\min}$) can be calculated without determining any more possible project scenarios and project structures.

This paper focuses on how to handle score values to specify a project scenario and project structures. It is assumed, that these score values are given. Since scoring methods may require more computational efforts than selecting the most desired project structure, in this paper a simple score value = probability value is used to characterize project scenarios and project structures. The following research may deal with how to join the proposed ranking and selection method to the scoring algorithms.

3.2.2. Calculation of possible project schedules

If the project structure is given and can be described as a DSM matrix, the total project time can be easily calculated (see Minogue et al., 2011), which is briefly outlined in this section. Let tpt be a

function of total project time (see a simple version of calculating TPT (Total Project Time = project duration) at the Pseudo code 1, in the Appendix A). In case of considering a deterministic PDM, $\mathbf{LD}$ is an n by n matrix of logic domain of PDM and in this way $\mathbf{TD}$ is an n by 1 vector of task durations. Though Pseudo code 1 shows how to calculate the duration of the project if there are only finish-to-start relations between tasks, DSM can handle different kinds of relations like start-to-start, start-to-finish, finish-to-finish (see Minogue et al., 2011). The proposed algorithm require only that the successor tasks cannot be finished before the predecessor tasks are finished. In this way the maximal value of the project duration ($T_{\max}$) will be performed when every (uncertain) task is also be completed and every dependency is realized. This DSM matrix is the ceiling matrix of $\mathbf{LD}$ ($\lceil \mathbf{LD} \rceil$), $T_{\max} = \text{tpt}(\lceil \mathbf{LD} \rceil, \mathbf{TD})$. By contrast the minimum value of the total project time ($T_{\min}$) is realized, when every uncertain task completion and every dependency is ignored. This DSM matrix is the floor matrix of $\mathbf{LD}$ ($\lfloor \mathbf{LD} \rfloor$), $T_{\min} = \text{tpt}(\lfloor \mathbf{LD} \rfloor, \mathbf{TD})$. It should be noticed, that $\lfloor \mathbf{LD} \rfloor$ and $\lceil \mathbf{LD} \rceil$ contains only 0 and 1 values, therefore, both $\lfloor \mathbf{LD} \rfloor$ and $\lceil \mathbf{LD} \rceil$ specify a (minimal/maximal time demanding) project structure.

This estimation similarly to the calculation of maximal probability value of a project structure can be determined without specifying any more possible project structures.

After deciding which tasks have to be included or excluded from the project plan, both the minimal and maximal values of the total project time can be specified.

Total project time can be determined, if the project network is acyclic graph, in other words DSM matrix of the project structure can be represented as an upper triangular matrix, however, a SDP may contain loop(s), which has to be resolved. Thus the Section 3.2.5. shows how to maintain feedbacks and loops.

In this section a deterministic specified PDM is considered, however, $\mathbf{TD}$ can also represent stochastic task durations. If the vector of minimal ($\mathbf{t}_{\min}$) and maximal ($\mathbf{t}_{\max}$) task durations can be estimated than the minimal ($T_{\min}$) and the maximal ($T_{\max}$) project durations can also be estimated as follows: $T_{\max} = \text{tpt}(\lceil \mathbf{LD} \rceil, \mathbf{t}_{\max})$, $T_{\min} = \text{tpt}(\lfloor \mathbf{LD} \rfloor, \mathbf{t}_{\min})$.

3.2.3. Calculation of possible direct costs

At the first case a deterministic specified PDM is considered. Let $\mathbf{CD}$ be the cost domain of PDM that is an n by 1 vector of (direct) cost of task completions $\mathbf{CD} = \{c_i | i = 1, 2, \dots, n\}$. The total project (direct) cost (TPC) is the sum of the tasks' (direct) costs.

Let tpc be a function of calculating TPC. $\mathbf{LD} = \{e_{ij} | i = 1, 2, \dots, n\}$ is an n by n matrix of logic domain. The $\text{tpc}(\mathbf{LD}, \mathbf{CD}) = \sum_{i=1, e_{ii}>0}^n c_i$ gives the possible direct cost of the project.

Let $\bar{\mathbf{LD}} = \{\bar{e}_{ij} | \bar{e}_{ij} = e_{ij}, i \neq j; \bar{e}_{ii} = \lceil e_{ii} \rceil\}$, $\underline{\mathbf{LD}} = \{\underline{e}_{ij} | \underline{e}_{ij} = e_{ij}, i \neq j; \underline{e}_{ii} = \lfloor e_{ii} \rfloor\}$ be n by n matrices. $\bar{\mathbf{LD}}/\underline{\mathbf{LD}}$ represents a project scenario, where every uncertain task completion included/excluded from the project. In this case the project cost will be maximal ($C_{\max}$)/minimal ($C_{\min}$). Without determining any more possible project scenario the maximal/minimal value of the project cost can be calculated as follows: $C_{\max} = \text{tpc}(\bar{\mathbf{LD}}, \mathbf{CD}) = \sum_{i=1, \bar{e}_{ii}>0}^n c_i$, $C_{\min} = \text{tpc}(\underline{\mathbf{LD}}, \mathbf{CD}) = \sum_{i=1, \underline{e}_{ii}>0}^n c_i$.

In this section a deterministic specified PDM is considered, however, $\mathbf{CD}$ can represent stochastic cost demands. If the vector of minimal ($\mathbf{c}_{\min}$) and maximal ($\mathbf{c}_{\max}$) cost demands can be estimated than the minimal ($C_{\min}$) and the maximal ($C_{\max}$) project cost can also be estimated as follows: $C_{\max} = \text{tpc}(\bar{\mathbf{LD}}, \mathbf{c}_{\max})$, $C_{\min} = \text{tpc}(\underline{\mathbf{LD}}, \mathbf{c}_{\min})$.

Time demands and cost demands may be depended each other (see time-cost trade-off methods (De et al., 1995; Feng et al., 2000), or a time-quality-cost trade-off methods (see e.g. Mungle et al., 2013)). Although $\mathbf{c}_{\max}$, $\mathbf{t}_{\max}$, $\mathbf{c}_{\min}$, $\mathbf{t}_{\min}$ can be determined, but the in case of time-quality-cost trade-off problems finding optimal

solution is an NP hard problem, next paper will introduce a matrix-based time-cost and matrix-based time-quality-cost trade-off problems.

3.2.4. Calculation of resource demands

Consider a specified deterministic PDM. Let $\mathbf{RD}$ be the resource domain of PDM that is an n by r matrix of (renewable) resources, where n is the number of tasks and r is the number of resources. $\mathbf{TD}$ is an n by 1 vector of task duration times. $\mathbf{LD} = \{e_{ij} | i = 1, 2, \dots, n\}$ is an n by n matrix of logic domain. Let $\text{maxres}(\mathbf{LD}, \mathbf{TD}, \mathbf{RD})$ be a function, which gives the maximal value of the resource demands (where every task will be scheduled to early start time).

Without counting any more possible project structure the maximal resource demand(s) occur if every uncertain task is realized, but every uncertain dependency is ignored ($\lfloor \mathbf{LD} \rfloor = \{\lfloor \bar{e}_{ij} \rfloor | \lfloor \bar{e}_{ij} \rfloor = \lfloor e_{ij} \rfloor, i \neq j; \lfloor \bar{e}_{ii} \rfloor = \lfloor e_{ii} \rfloor\}$).

$$\mathbf{R}_{\max} = \text{maxres}(\lfloor \mathbf{LD} \rfloor, \mathbf{TD}, \mathbf{RD}).$$

The minimal resource demands occurred if every uncertain task is ignored, but every uncertain dependency is realized:

$$\lceil \mathbf{LD} \rceil = \{\lceil \bar{e}_{ij} \rceil | \lceil \bar{e}_{ij} \rceil = \lceil e_{ij} \rceil, i \neq j; \lceil \bar{e}_{ii} \rceil = \lceil e_{ii} \rceil\}.$$

$\mathbf{R}_{\min} = \text{maxres}(\lceil \mathbf{LD} \rceil, \mathbf{TD}, \mathbf{RD})$, where $\mathbf{R}_{\max}$, $\mathbf{R}_{\min}$ are r by 1 vector of highest resource demand(s).

Similarly to the stochastic cost demands, stochastic resource demands can also be considered and the dependency between time and resources can be considered via resource-time and resource-quality-time trade-off methods. Since most of these methods are NP-hard problems, next paper may deal with matrix-based complex project planning methods.

3.2.5. Handling cycles

If uncertain dependencies can be interpreted as probability values similarly to the GERT (Graphical Evaluation and Review Technique) method (Pritsker, 1966) cycles can be resolved. Based on to the Uma Mahenswari's algorithm (Uma, Varghese, & Sridharan, 2006), the first step is to detect cycles and sequencing the SNPM matrix. There is a sequencing method for (numerical) DSM. If this approach is applied the numerical value (in this case the probability value) of backward loop will be minimal. The second step is to eliminate backward loops and recalculate time and cost demands of the feedbacked tasks.

After eliminating cycles the expected value of duration of feedbacked task (t') and direct cost (c') can be estimated as follows: $t' = t + pt + p^2t + \dots = \sum_{i=0}^{\infty} p^i t = t/(1-p)$, $c' = c + pc + p^2c + \dots = \sum_{i=0}^{\infty} p^i c = c/(1-p)$, where p is the probability value of the backward loop; t and c are the duration and the cost demands of feedbacked task, without considering the loop. The maximal resource demands assumed as time invariants.

This method assumes that feedback can occur any number of times with getting lower and lower probability in each new cycle (of occurrence). However, usually the numbers of iterations are finite. Thus Shi and Blomquist (2012) approach should be used for estimating the project duration.

3.3. Selecting and ranking project scenarios and project structures

One of the most important feature of the proposed ageratum: there is no need to determine any possible project scenario and project structure in order to determine the possible maximal and minimal score values. The Fig. 3. shows the maximal and minimal score values for a given specified deterministic PDM.

The following subsection shows how to exclude infeasible project structures and project scenarios. After discussing the possible constraints and objective functions a fast algorithm will

$P_{\max}=0.8*(1-0.1)*0.7=0.504$	PDM	A	B	C	D	E	t	c	r_1	r_2	r_3	$R_{1\max}=11$
$P_{\min}=(1-0.8)*0.1*(1-0.7)=0.006$	A	0.8	1	0.8	0.2	0.1	4	2.4	2	1	1	$R_{1\min}=3$
$T_{\max}=4+\max(2;4)+3=11$	B		1		0.4	0.9	2	1.8	3		3	$R_{2\max}=7$
$T_{\min}=2$	C			0.1		0.1	4	9.6	8	2	5	$R_{2\min}=0$
$C_{\max}=2.4+1.8+9.6+0.9=14.7$	D				0	0.3	10	42	12	3	2	$R_{3\max}=8$
$C_{\min}=1.8$	E					0.7	3	0.9	1	4	2	$R_{3\min}=3$
	Logic Domain (LD)						TD	CD	Resource Domain (RD)			

Fig. 3. A specified deterministic PDM with calculated maximal and minimal values of scores.

be introduced in order to demonstrate how to find the optimal solution.

3.3.1. Constraints and objective functions

Regarding the score values, different kinds of algorithms can be defined. One can determine the most probable, less time or less cost demanded project plan with respect to the constraints. First, we have to define the objective functions:

- (1) Specifying most probable (most desired) project scenarios and structures.
- (2) Specifying shortest project plan.
- (3) Specifying project plan with smallest budget.

- More than one objective function can be combined, however, in this case the rank of the objective functions must be defined.

Other complex objective function is not considered in this paper. After specifying (at least) one objective function constraint(s) can be specified.

Similarly to the objective functions the constraints can be:

- (1) Score value of a project scenario (C_p) or a project structure (C_p);
- (2) Time constraint (C_t);
- (3) Resource constraints (C_r);
- (4) Project budget (C_c).

The second key feature is that the infeasible project scenarios and project structures can be excluded by regarding the following rules.

- (1) $C_p > P_{\max}$: If the score constraint of project scenario (C_p) is greater than the maximal score (in this case probability) value ($P_{\max}$) of the project scenario, than the project scenario is infeasible. Therefore, all project structures specified from this project scenario are also infeasible. We can say there are no project scenarios to reach the desired probability value.
- (2) $C_p > p_{\max}$: If the score constraint of project structure (C_p) is greater than the maximal score (in this case probability) value ($p_{\max}$) of the project structure, than the project structure is infeasible. We can say there are no project structures to achieve the desired probability value.
- (3) $C_t < T_{\min}$: If the time constraint is lower than the minimal value of possible project duration time, than the given project scenario/the given project structure is infeasible. We can say the given project scenario/the given project structure cannot be completed within the given time interval.
- (4) $C_c < C_{\min}$: If the cost constraint is lower than the minimal value of possible cost demands, than the given project scenario is infeasible. Therefore, all project structures specified

from this project scenario are also infeasible. We can say this given project scenario cannot be completed within the given budget.

One can notice there is no rule for resource demands. Since if $R_{\min}$ is not lower than the vector of resource constraint C_r , then a resource allocation or a resource levelling algorithms (see Brucker et al., 1999) can solve this problem, therefore resource allocation will be respected after specifying project plans.

3.3.2. Finding best project scenario and project structures

Let us assume that there are no constraints. The goal is to find best = most probable or less time/cost demanding project scenarios or project structures.

The proposed method is a tree traversal algorithm along with we perform book-keeping of the probability/time/cost values. The decision about task realization we always have two options include or exclude. In the binary decision tree comparing the score values we should decide a task or a dependency will be included or excluded.

This greedy algorithm specifies the most probable project scenario in t steps, where t is the number of uncertain task completions (see Fig. 4.).

After every decision the maximal conditional probability values can be determined. Since we always select the best value in every decision situation the most probable project scenario (see Fig. 4.), most probable project structure (see the BestDSM function in Pseudo code 2, in the Appendix A) and similarly finding the less time demanded project structure or specifying the less cost demanded project scenario can be calculated within $O(k)$, where k is the number of uncertain cells in LD of a PDM.

3.3.3. Finding the top n project structures and project scenarios

When finding the most probable project scenario or the most probable project structure, maximal probability values should be stored into a sorted **buffer** set. Elements of this set are descendant ordered by score values. The next value of this set will be the probability of next most probable project scenario/next most probable project structure. Thus, after specifying most probable/most desired project plans the next most probable/next most desired project plans can also be determined.

According to the score values the decision tree is a special heap. In this case it means $P_{\max}, p_{\max}, T_{\max}, C_{\max}$ of child nodes are not greater than the score values of a given parent node. Similarly for the minimal values: $P_{\min}, p_{\min}, T_{\min}, C_{\min}$ of child nodes are not lower than the score values of a given parent node. In general the n^{th} largest value can be determined within $O(n \log n)$ computation time (see e.g. Sack & Strothotte, 1990), however, this decision tree is a special binary heap, where quasi linear search algorithm can be specified.

Let p be the given probability value of a project structure, which has to be found in the decision tree. Every node in the decision tree

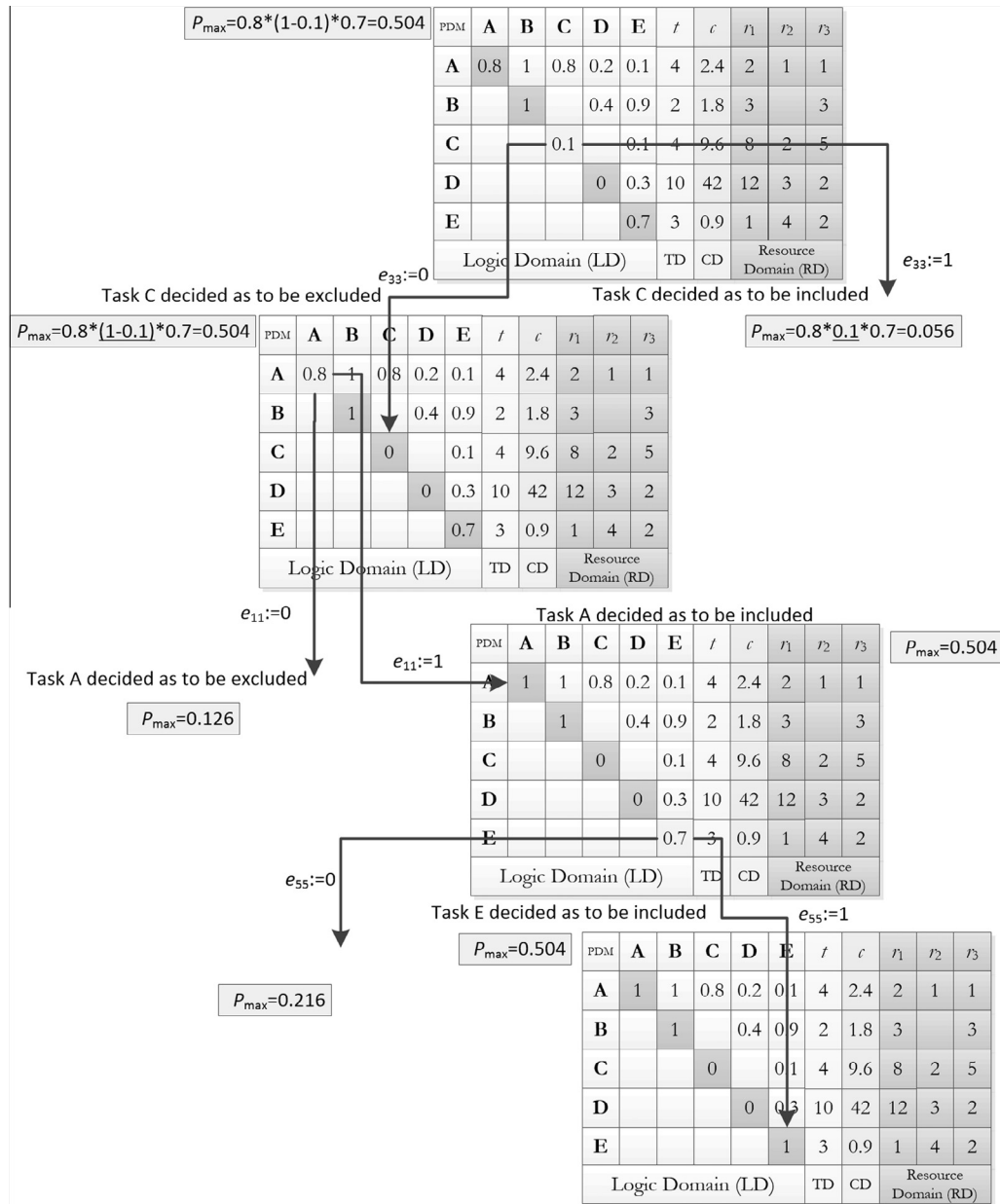


Fig. 4. Specifying most probable project scenario.

represents a specified deterministic PDM matrix, where the maximal and minimal possible value of the probabilities can be determined without specifying the possible child nodes. If a parent node is infeasible, then all child nodes, and all child nodes of the child nodes are infeasible. For example, in this case, according to the characteristics of the decision tree, if the maximal probability value of the (parent) node is lower than p ($P_{\max} < p$, $p_{\max} < p$), then child nodes of this parent node should be excluded because maximal conditional probability values of child nodes cannot be higher than the maximal score value of the parent node. Similarly if the minimal probability value of the (parent) node is greater than p ($P_{\min} > p$, $p_{\min} > p$), then child nodes of this parent node should be excluded.

When finding the most probable project structure we will store the next greatest probability value and it will be found at the next step, when the third greatest probability value will be stored.

In this way the top n project structures (see the Pseudo code 3, in the Appendix A) or similarly the top n project scenarios can be specified.

3.3.4. Expert Project Ranking (EPR) algorithm

The proposed Expert Project Ranking (EPR) algorithm is based on the formerly proposed methods and recognitions. This algorithm will find the top n project structure within the given constraints.

The following properties of the decision trees will be applied:

- Every node represents a specific deterministic PDM
- The binary tree is perfect, because every node other than the leaves, except possibly the last, has two children (include or exclude task completion/task dependency)
- If a specific deterministic PDM is infeasible represented by a (parent) node, then every child node of the parent node is also infeasible.
- With respect to the score values, the binary tree is a binary heap.
- Maximal and minimal possible score values can be calculated without specifying child nodes.

- However, the maximal/minimal score value can be found by a tree traversal algorithm.
- Last nodes give us the project scenarios and project structures.

At the first step we have to specify the objective function and constraints (see Section 3.3.1). The Pseudo code 4 in the Appendix A gives the top n project structures, where a specified deterministic PDM (with **LD**, **TD**, **CD**, **RD** domains) is given with respect to the time/cost/resource constraints: C_t , C_c , C_r .

This algorithm is a generic algorithm. Other objective functions (like least TPT, least TPC) and other constraints can be defined instead of cost, time and resource constraint. In the next section simulated project scenarios and project structures are presented in order to test the EPR-method in large, complex projects.

4. Algorithmic complexity

This section shows how the EPR is scaling with the number of entities. If there is no uncertain task completion, the number of possible project structure depends only on the number of uncertain dependencies. If there are k uncertain dependencies then there are 2^k possible project plans. The project network can be specified as an SNPM matrix. In case of acyclic project networks the maximal number of uncertain dependency is $n(n-1)/2$, and in this case the number of possible project structures are: $2^{n(n-1)/2}$. If there are t uncertain tasks, then 2^t project scenarios can be specified. In special case if $t = n$, and $k = n(n-1)/2$ then there are 2^n project scenarios and $\sum_{j=0}^n \binom{n}{j} 2^{j(j-1)/2}$ project plans.

It is easy to build a matrix, which specifies huge numbers of project scenarios and project structures: i.e. when a 10 by 10 PEM, which can be a logic domain of a PDM, matrix contains $t = n = 10$ uncertain tasks and $k = n(n-1)/2$ uncertain dependencies, the number of project scenarios is more than $3,588 \cdot 10^{13}$. In the first simulation task, different sizes of SNPM and PEM matrices were specified by fulfilled upper triangular matrices. The goal was to select the top 1, 10, 100 and 1000 project plans, where score values of project plans are decreasingly sorted. Uncertainty value of dependencies are in $(0, 1)$ (open) interval follows uniform distribution. If PEM contains uncertain tasks and uncertain task dependencies, the number of possible project plans are very similar. Let $s = 2^k = 2^{n(n-1)/2}$ be the number of possible project plans in case of considering fulfilled upper triangular n by n SNPM matrix; $p = \sum_{j=0}^n \binom{t}{j} 2^{j(j-1)/2} = \sum_{j=0}^n \binom{n}{j} 2^{j(j-1)/2}$ be the number of possible project plans in case of considering fulfilled upper triangular n by n PEM matrix, where every task completion is also uncertain. Since $\lim_{n \rightarrow \infty} \sqrt{\log_2 s} = \lim_{n \rightarrow \infty} \sqrt{\log_2 p} = \sqrt{2}(\frac{n}{2} - \frac{1}{4})$ the complexity of specifying project plans from SNPM and PEM matrices are similar (logarithmic value of the convergence of complexity is identical). The computational demand of EPR algorithm with respect to the fulfilled PEM = LD of a specified deterministic PEM matrix is $O(k) = O(n(n-1)/2) \approx O(n^2)$ in case of considering fulfilled upper triangle SNPM matrix, and the run time $O(k+t) = O(n + n(n-1)/2) \approx O(n^2)$ is also similar when regarding a fulfilled upper triangle PEM matrix. In case of $n = 50$ a fulfilled upper triangle SNPM and PEM matrices specify $5,78 \cdot 10^{368}$ possible project plans.

In the second simulation cost and time demands are randomized and followed uniform distributions. The goal were to specify the top 10 most desired/most probable project plans from a 50 by 50 fulfilled PEM matrix, where every uncertainty value of task completion and dependency was the $(0.5, 1)$ (open) intervals, which means every tasks and every dependencies should be

Table 4

Simulation results.

$T_{\max}$	90%		85%		80%	
$C_{\max}$	P%	p%	P%	p%	P%	p%
90%	10.2	0.5	10.4	2.4	10.3	3.5
85%	14.9	0.1	15.1	0.6	15.4	1.2
80%	19.9	0.0	20.1	0.1	19.8	0.4

realized (the probability of including (task or a task dependency) was greater than the probability of excluding).

After specifying PDMs, the maximal values of the project duration ($T_{\max}$) and the maximum cost demand ($C_{\max}$) can be determined, without specifying the project scenarios and project structures. In this simulation the agile approach is considered. The goal was to find the most probable project scenario and most probable project structures, since the probability values were greater than 0.5 the most probable project scenario include all tasks. The question was that how many tasks should be excluded in order to satisfy the cost constraint.

Cost and time demands of the project was decreased by 10, 15 and 20%. This simulation demonstrates how to restructure or reorganize the project if we decrease the time and cost constraints. Table 4 shows that, if we decrease the budget ($C_c = 0.9C_{\max}, \dots, C_c = 0.8C_{\max}$) and the project duration ($C_t = 0.9T_{\max}, \dots, C_t = 0.8T_{\max}$), how many tasks (P%) will be excluded and how many dependencies (p%) will be resolved.

While excluding tasks may also reduce the project duration, the claim for restructuring the project is minimal. However, different kinds of projects may require different kinds of intervention. It will be discussed in the next paper, where the focus will be on the matrix-based risk analysis and risk management techniques.

5. Summary and conclusion

Agile project management grew out of a need to manage projects characterized by complexity and uncertainty with responsiveness and adaptability. When goals and solutions are unclear and there is high volatility, there is particular need for alternative approaches to managing projects.

Network-based project planning and resource allocation methods primarily support the traditional project management (TPM) approach, however, agile project management (APM) requires new project planning methods.

In this paper a new matrix-based project planning method was proposed for describing possible time/cost/resource demands and possible project scenarios and project structures. Since there is huge number of variations of different kinds of project scenarios and project structures, an efficient algorithm is needed for selecting adequate project scenarios and project structures considering the management claims.

The proposed Expert Project Ranking (EPR) algorithm is a fast, efficient method and it supports the agile project management (APM) approach. The algorithm is able to rank all the feasible project plans according to the predefined preferences of scores like time, cost and resources.

The developed matrix based methods and proposed exact algorithm may be important and essential components of a project expert system supporting strategic decision makings particularly in case of large, complex flexible projects.

5.1. Limitations

This method can model the uncertain task realization and task dependencies. However, if the technology or the project

```

1 function TPT=tpt(DSM,t):
2   n:=|t| //n: number of elements of vector t
3   EST:=0 //Earliest Start Time
4   EFT:=t //Earliest Finish Time
5   for i:=1 to (n-1):
6     if DSM(i,i)>0: //if task may be completed
7       for j:=i+1 to n:
8         if DSM(j,j)>0 and DSM(i,j)>0: //If there is an uncertain task dependency
9           if EFT(i)>EST(j):
10             EST(j):=EFT(i)
11             EFT(j):=EST(j)+t(j)
12           end
13         end
14       end
15     end
16   end
17   TPT:=max(EFT)
18

```

Pseudo code 1. Calculation of duration for a given project structure.

```

1 function DSM:=BestDSM(SNPM,p,q):
2   u:=false //Existence of uncertainty value
3   for i from 1 to n:
4     for j from i+1 to n:
5       if SNPM(i,j)<1 and SNPM(i,j)>0 and i<>j then: //If there are uncertain dep.
6         u:=true
7         SNPM0(i,j):=0 //Left side
8         maxs0:=maxscore(SNPM0,p,q);
9         SNPM1(i,j):=0 //Right side
10        maxs1:=maxscore(SNPM1,p,q);
11        if maxs0>maxs1 then: //Go to find maximal score value.
12          DSM:=BestDSM(SNPM0,p,q)
13        else: DSM:=BestDSM(SNPM1,p,q)
14        return
15      if u=false then: DSM:=SNPM//Evaluated SNPM will be the best DSM
16    return DSM

```

Pseudo code 2. Calculation of highest resource demands for a given project structure.

```

1 function DSMs:=DSMfind(SNPM,DSMs,p,q,score):
2   [i,j]=argmax(p,q)where 0<SNPM<1 //i,j will be the place where score value is maximal
3   if i≠0 and j≠0:
4     SNPM(i,j):=0 //left side
5     pmax0:maxscore(SNPM,p,q)
6     pmin0:minscore(SNPM,p,q)
7     if pmax0>score and pmin0<=score then:
8       DSMs:=DSMfind(SNPM,DSMs,p,q,score)
9     else: //store maxscore value into the buffer set
10      if pmax0∈buffer then: buffer=insertdescend(pmax0)
11      SNPM(i,j):=1 //right side
12      pmax1:maxscore(SNPM,p,q)
13      pmin1:minscore(SNPM,p,q)
14      if pmax1>score and pmin1<=score then:
15        DSMs:=DSMfind(SNPM,DSMs,p,q,score)
16      else: //store maxscore value into the buffer set
17        if pmax1∈buffer then: BUFFER=insertdescend(pmax1)
18      else: DSMs:=DSMs∪SNPM //Storing the evaluated SNPM into DSMs
19    return DSMs

20 function DSMs=snpm2dsmranked(SNPM,p,q,n) //specify top n project structure
21   global buffer
22   buffer:=maxscore(SNPM,p,q)
23   i:=1
24   do
25     DSMs=DSMs∪DSMfind(SNPM,∅,p,q,buffer(i))
26     i:=i+1
27   while i<=min(numel(buffer),n):
28   return

```

Pseudo code 3. Specification of top n project structures.

characteristic prohibit flexible dependencies and task exclusions, we cannot enjoy the benefits of the proposed algorithm. Nonetheless, this approach may be useful at the next level, where we

specifying a multi-project or a project portfolios. Nevertheless, this paper focuses only on the specification of an adequate project plan. If we would like extend this approach for multi-project project

```

1 function DSMs:=DSMfind(SNPM,DSMs,p,q,score,t,r,CTIME,CRES):
2   [i,j]=argmax(p,q)where 0<SNPM<1 //i,j will be the place where score value is maximal
3   if i≠0 and j≠0:
4     SNPM(i,j):=0 //left side
5     PMAX0:=maxscore(SNPM,p,q)
6     PMIN0:=minscore(SNPM,p,q)
7     tMIN0:=tpt(LSNPM,t)
8     if PMAX0>=score and PMIN0<=score and tMIN0<=CTIME then:
9       DSMs:=DSMfind(SNPM,DSMs,p,q,score,t,r,CTIME,CRES)
10    else: //store maxscore value into the buffer set
11      if PMAX0∈buffer then: buffer=insertdescend(PMAX0)
12      SNPM(i,j):=1 //right side
13      PMAX1:=maxscore(SNPM,p,q)
14      PMIN1:minscore(SNPM,p,q)
15      tMIN1:tpt(LSNPM,t)
16      if PMAX1>=score and PMIN1<=score and tMIN1<=CTIME then:
17        DSMs:=DSMfind(SNPM,DSMs,p,q,score,t,r,CTIME,CRES)
18      else: //store maxscore value into the buffer set
19        if PMAX1∈buffer then: buffer=insertdescend(PMAX1)
20    else: if RALL(SNPM,CRES,CTIME,t,r) is feasible than : DSMs:=DSMs∪SNPM
21  return DSMs

22 function SNPMs:=SNPMfind(PEM,SNPMs,P,Q,SCORE,T,R,C,CTIME,CCOST):
23  i=argmax(diag(P),diag(Q))where 0<diag(PEM)<1
24  if i≠0:
25    PEM(i,i):=0 //left side
26    PMAX0:MAXSCORE(PEM,P,Q) : PMIN0:MINSORE(PEM,P,Q)
27    CMIN0:tpc(PEM,C)
28    TMIN0:tpt(LPEM,t)
29    if PMAX0>=SCORE and PMIN0<=SCORE and CMIN0<=CCOST and TMIN0<=CTIME then:
30      SNPMs:=SNPMfind(PEM,SNPMs,P,Q,SCORE,T,R,C,CTIME,CCOST)
31    else: //store MAXSCORE value into the BUFFER set
32      if PMAX0∈BUFFER then: BUFFER=insertdescend(PMAX0)
33      PEM(i,i):=1 //A bizonytalan tevékenység értéke legyen 1.
34      PMAX1:MAXSCORE(PEM,P,Q) : PMIN1:MINSORE(PEM,P,Q)
35      CMIN1:tpc(PEM,C)
36      TMIN1:tpt(LPEM,t)
37      if PMAX1>=SCORE and PMIN1<=SCORE and CMIN1<=CCOST and TMIN1<=CTIME then:
38        SNPMs:=SNPMfind(PEM,SNPMs,P,Q,SCORE,T,R,C,CTIME,CCOST)
39      else: //store MAXSCORE value into the BUFFER set
40        if PMAX1∈BUFFER then: BUFFER=insertdescend(PMAX1)
41    else: SNPMs:=SNPMs∪PEM
42  return SNPMs

43 function DSMs=EPR(PEM,P,Q,T,R,C,CTIME,CCOST,CRES,n):
44  global BUFFER,buffer
45  BUFFER:=MAXSCORE(PEM,P,Q)
46  SNPMs:=∅
47  DSMs:=∅
48  i:=1
49  solutions:=0
50  do
51    SNPMs=SNPMs∪SNPMfind(PEM,SNPMs,P,Q,BUFFER(i),T,R,C,CTIME,CCOST)
52    [SNPM,p,q,t,r]=delete_empty_diag_cells(SNPMs(i),P,Q,T,R)
53    buffer:=maxscore(SNPM,p,q)
54    j:=1
55    do
56      DSMs=DSMs∪DSMfind(SNPM,DSMs,p,q,buffer(j),t,r,CTIME,CRES)
57      j:=j+1
58      solutions:=solutions+1
59    while j<=numel(buffer) and solutions<=n:
60      i:=i+1
61    while i<=numel(BUFFER):
62  return DSMs

```

Pseudo code 4. Expert Project Ranking (EPR) method for finding most probable project structure within a most probable project scenario with respect to the time, resource and cost constraints.

selection and specifying project portfolios, a scoring method (like, AHP, ER, ER + AHP etc.) should be included.

This algorithm is not focusing on the time–cost and the time–cost–quality trade off problems, however, applying this method, restructuring and reorganizing the project with respect to the agile project management approach both time and cost demand can be decreased. Integrating EPR method to the time–cost and time–quality–cost trade off methods can be discussed in a forthcoming article.

5.2. Future works

In this paper three new further directions are proposed. EPR focused on how to select a proper project plan, if the claim of the task realizations and the technological dependency is not explicitly specified. This approach can be extended to manage multi-projects and project portfolios. In this case the proposed method may be integrated with project scoring techniques.

The proposed EPR method can simultaneously decrease the time and the cost demands, however, it will be interesting to compare to or integrate with the traditional time–cost trade-off and/or time–quality–cost trade off methods.

The other possible application of this method is in risk management and risk analysis. Uncertain task completion may model the changes of management or client claims. Uncertain task dependency may model the technological changes. In this case a more appropriate matrix-based model can be specified.

Appendix A. Appendix – Pseudo codes

Pseudo codes 1–4.

References

- Alba, E., & Chicano, J. F. (2007). Software project management with {GAs}. *Information Sciences*, 177(11), 2380–2401.
- Arditi, D., & Patel, B. K. (1989). Expert system for claim management in construction projects. *International Journal of Project Management*, 7(3), 141–146.
- Badukale, P. A., & Sabihuddin, S. (2014). Article: Line of balance. *International Journal of Modern Engineering Research*, 4(3), 45–47.
- Baker, N., & Freeland, J. (1975). Recent advances in r&d benefit measurement and project selection methods. *Management Science*, 21(10), 1164–1175.
- Boehm, B. W., Madachy, R., & Steece, B. (2000). *Software cost estimation with cocomo II*. Prentice Hall PTR.
- Browning, T., & Eppinger, S. (2002). Modeling impacts of process architecture on cost and schedule risk in product development. *IEEE Transactions on Engineering Management*, 49(4), 428–442.
- Browning, T. R. (2014). Managing complex project process models with a process architecture framework. *International Journal of Project Management*, 32(2), 229–241.
- Brucker, P., Drexel, A., Mohring, R., Neumann, K., & Pesch, E. (1999). Resource-constrained project scheduling: Notation, classification, models, and methods. *European Journal of Operational Research*, 112(1), 3–41.
- Burger, R., & Fischer, M. (1985). An expert system for project organization. In *INTERNET seminar*.
- Calantone, R. J., Di Benedetto, C. A., & Schmidt, J. B. (1999). Using the analytic hierarchy process in new product screening. *Journal of Product Innovation Management*, 16(1), 65–76.
- Carbonell, P., Rodriguez Escudero, A., & Munuera Aleman, J. (2004). Technology newness and impact of go/no-go criteria on new product success. *Marketing Letters*, 15(2–3), 81–97.
- Chan, F. K., & Thong, J. Y. (2009). Acceptance of agile methodologies: A critical review and conceptual framework. *Decision Support Systems*, 46(4), 803–814. [IT] Decisions in Organizations.
- Chang, C. K., Jiang, H.-y., Di, Y., Zhu, D., & Ge, Y. (2008). Time-line based model for software project scheduling with genetic algorithms. *Information and Software Technology*, 50(11), 1142–1154.
- Chen, C.-H., Ling, S. F., & Chen, W. (2003). Project scheduling for collaborative product development using [DSM]. *International Journal of Project Management*, 21(4), 291–299.
- Chen, R.-M. (2011). Particle swarm optimization with justification and designed mechanisms for resource-constrained project scheduling problem. *Expert Systems with Applications*, 38(6), 7102–7111.
- Chin, K.-S., Xu, D.-L., Yang, J.-B., & Ping-Kit Lam, J. (2008). Group-based er-ahp system for product project screening. *Expert Systems with Applications*, 35(4), 1909–1929.
- Cho, S. (2006). An exploratory project expert system for eliciting correlation coefficient and sequential updating of duration estimation. *Expert Systems with Applications*, 30(4), 553–560.
- Coldrick, S., Longhurst, P., Ivey, P., & Hannis, J. (2005). An r&d options selection model for investment decisions. *Technovation*, 25(3), 185–193.
- Crawford, L., Pollack, J., & England, D. (2006). Uncovering the trends in project management: Journal emphases over the last 10 years. *International Journal of Project Management*, 24(2), 175–184.
- Dalcher, D. (2009). Managing complex projects: A new model. *Project Management Journal*, 40(3), 83.
- Danilovic, M., & Browning, T. R. (2007). Managing complex product development projects with design structure matrices and domain mapping matrices. *International Journal of Project Management*, 25(3), 300–314.
- De, P., Dunne, E. J., Ghosh, J. B., & Wells, C. E. (1995). The discrete time–cost tradeoff problem revisited. *European Journal of Operational Research*, 81(2), 225–238.
- Dingsøyr, T., Nerur, S., Balijepally, V., & Moe, N. B. (2012). A decade of agile methodologies: Towards explaining agile software development. *Journal of Systems and Software*, 85(6), 1213–1221. Special Issue: Agile Development.
- Eisner, H. (1962). A generalized network approach to the planning and scheduling of a research project. *Operations Research*, 10(1), 115–125.
- Eppinger, S., Whitney, D., Smith, R., & Gebala, D. (1994). A model-based method for organizing tasks in product development. *Research in Engineering Design*, 6(1), 1–13.
- Evaristo, R., & van Fenema, P. C. (1999). A typology of project management: Emergence and evolution of new forms. *International Journal of Project Management*, 17(5), 275–281.
- Fekri, R., Aliahmadi, A., & Fathian, M. (2009). Identifying the cause and effect factors of agile npd process with fuzzy dematel method: The case of iranian companies. *Journal of Intelligent Manufacturing*, 20(6), 637–648.
- Feng, C., Liu, L., & Burns, S. (2000). Stochastic construction time–cost trade-off analysis. *Journal of Computing in Civil Engineering*, 14(2), 117–126.
- Gidel, T., Gautier, R., & Duchamp, R. (2005). Decision-making framework methodology: an original approach to project risk management in new product design. *Journal of Engineering Design*, 16(1), 1–23.
- Hadad, Y., Keren, B., & Laslo, Z. (2013). A decision-making support system module for project manager selection according to past performance. *International Journal of Project Management*, 31(4), 532–541.
- Hart, S., Jan Hultink, E., Tzokas, N., & Commandeur, H. R. (2003). Industrial companies' evaluation criteria in new product development gates. *Journal of Product Innovation Management*, 20(1), 22–36.
- Hashim, N. I., Chileshe, N., & Baroudi, B. (2013). Management challenges within multiple project environments: Lessons for developing countries. In *Australasian journal of construction economics and building-conference series* (Vol. 1, pp. 21–31).
- Hsu, Y.-G., Tzeng, G.-H., & Shyu, J. Z. (2003). Fuzzy multiple criteria selection of government-sponsored frontier technology r&d projects. *R&D Management*, 33(5), 539–551.
- Kastor, A., & Sirakoulis, K. (2009). The effectiveness of resource levelling tools for resource constraint project scheduling problem. *International Journal of Project Management*, 27(5), 493–500.
- Kelley, J. E. (1961). Critical-path planning and scheduling: Mathematical basis. *Operations Research*, 9(3), 296–320.
- Kiss, J. (2013). Next generational applications supporting the planning phase of projects. *Global Journal on Technology*, 3, 343–348.
- Kosztýán, Z. T. (2012). Challenges of the project planning methods in the 21st century. *Problems of Management in the 21st Century*, 5, 46–60.
- Kosztýán, Z. T., & Kiss, J. (2010a). Pem – a new matrix method for supporting the logic planning of software development projects. In *DSM 2010: Proceedings of the 12th international DSM conference*. Cambridge, UK, 22–23.07. 2010.
- Kosztýán, Z. T., & Kiss, J. (2010b). Stochastic network planning method. In K. Elleithy (Ed.), *Advanced techniques in computing sciences and software engineering* (pp. 263–268). Netherlands: Springer.
- Kosztýán, Z. T., & Kiss, J. (2013). Matrix-based methods for supporting logic planning of it projects. In *Emerging trends in computing, informatics, systems sciences, and engineering* (pp. 311–318). Springer.
- Krishnan, V., & Ulrich, K. T. (2001). Product development decisions: A review of the literature. *Management Science*, 47(1), 1–21.
- Laaloui, Y., & Bouguila, N. (2014). Pre-run-time scheduling in real-time systems: Current researches and artificial intelligence perspectives. *Expert Systems with Applications*, 41(5), 2196–2210.
- Liao, S.-H. (2005). Expert system methodologies and applications – a decade review from 1995 to 2004. *Expert Systems with Applications*, 28(1), 93–103.
- Linton, J. D., Walsh, S. T., & Morabito, J. (2002). Analysis, ranking and selection of r&d projects in a portfolio. *R&D Management*, 32(2), 139–148.
- Lootsma, F. A. (1997). *Fuzzy logic for planning and decision making*. London: Kluwer Academic Publishers.
- Mahdi, I. M., & Alreshaid, K. (2005). Decision support system for selecting the proper project delivery method using analytical hierarchy process (ahp). *International Journal of Project Management*, 23(7), 564–572.
- Minogue, P. et al. (2011). gantt-like dsms. In *DSM 2011: Proceedings of the 13th international DSM conference*.
- Mohanty, R. P., Agarwal, R., Choudhury, A. K., & Tiwari, M. K. (2005). A fuzzy anp-based approach to r&am:d project selection: A case study. *International Journal of Production Research*, 43(24), 5199–5216.
- Monghasemi, S., Nikoo, M. R., Fasaee, M. A. K., & Adamowski, J. (2015). A novel multi criteria decision making model for optimizing time–cost–quality trade-off problems in construction projects. *Expert Systems with Applications*, 42(6), 3089–3104.
- Mungle, S., Benyoucef, L., Son, Y.-J., & Tiwari, M. (2013). A fuzzy clustering-based genetic algorithm approach for time–cost–quality trade-off problems: A case study of highway construction project. *Engineering Applications of Artificial Intelligence*, 26(8), 1953–1966.
- Nan, N., & Harter, D. (2009). Impact of budget and schedule pressure on software development cycle time and effort. *IEEE Transactions on Software Engineering*, 35(5), 624–637.
- Oh, J., Yang, J., & Lee, S. (2012). Managing uncertainty to improve decision-making in (NPD) portfolio management with a fuzzy expert system. *Expert Systems with Applications*, 39(10), 9868–9885.
- Özdanar, L., & Ulusoy, G. (1995). A survey on the resource-constrained project scheduling problem. *IIE Transactions*, 27(5), 574–586.
- Pritsker, A. A. B. (1966). GERT: Graphical evaluation and review technique. Rand Corporation.
- Rahimian, V., Ramsin, R. (2008). Designing an agile methodology for mobile software development: A hybrid method engineering approach. In *Second international conference on research challenges in information science, 2008. RCIS 2008* (pp. 337–342).
- Rochford, L. (1991). Generating and screening new products ideas. *Industrial Marketing Management*, 20(4), 287–296.

- Roy, B. (1962). Graphes et ordonnancements. Note de Travail. SEMA (Metra International), Direction Scientifique.
- Sack, J.-R., & Strothotte, T. (1990). A characterization of heaps and its applications. *Information and Computation*, 86(1), 69–86.
- Shi, Q., & Blomquist, T. (2012). A new approach for project scheduling using fuzzy dependency structure matrix. *International Journal of Project Management*, 30(4), 503–510.
- Steward, D. V. (1981). The design structure system – a method for managing the design of complex systems. *IEEE Transactions on Engineering Management*, 28(3), 71–74.
- Tang, D., Zhu, R., Tang, J., Xu, R., & He, R. (2010). Product design knowledge management based on design structure matrix. *Advanced Engineering Informatics*, 24(2), 159–166. Enabling Technologies for Collaborative Design.
- Uma, M. J., Varghese, K., & Sridharan, T. (2006). Application of dependency structure matrix for activity sequencing in concurrent engineering projects. *Journal of Construction Engineering and Management*, 132(5), 482–490.
- Wang, Y.-M., & Elhag, T. M. (2008). Evidential reasoning approach for bridge condition assessment. *Expert Systems with Applications*, 34(1), 689–699.
- Wiest, J. D. (1981). Precedence diagramming method: Some unusual characteristics and their implications for project managers. *Journal of Operations Management*, 1(3), 121–130.
- Williams, L., & Cockburn, A. (2003). Guest editors' introduction: Agile software development: It's about feedback and change. *Computer*, 36(6), 39–43.
- Wilson, J. M. (2003). Gantt charts: A centenary appreciation. *European Journal of Operational Research*, 149(2), 430–437.
- Wysocki, R. K. (2009). *Effective Project Management: Traditional, Agile, Extreme*. John Wiley & Sons. 5. auflage edition.
- Xiao, R., Chen, T., & Tao, Z. (2007). Information modeling and reengineering for product development process. *International Journal of Management Science and Engineering Management*, 2(1), 64–74.
- Yan, H.-S., Wang, Z., & Jiang, M. (2002). A quantitative approach to the process modeling and planning in concurrent engineering. *Concurrent Engineering*, 10(2), 97–111.
- Yang, J.-B. (2001). Rule and utility based evidential reasoning approach for multiattribute decision analysis under uncertainties. *European Journal of Operational Research*, 131(1), 31–61.
- Yassine, A., Falkenburg, D., & Chelst, K. (1999). Engineering design management: An information structure approach. *International Journal of Production Research*, 37(13), 2957–2975.